

Résumé de l'Implémentation

✓ Tâches Accomplies

1. Architecture en 3 Couches ✓

Couche 1 - Détection

- ✓ Service de détection unifié (`DetectionService`)
- ✓ Détection regex avec validation (emails, téléphones, NIR, IBAN, etc.)
- ✓ Détection NER GLiNER (support GPU automatique)
- ✓ GPU optimizer intégré
- ✓ Déduplication intelligente (priorité: regex > ner-gpu > ner)

Couche 2 - Transformation

- ✓ Service de transformation unifié (`TransformationService`)
- ✓ Remplacement par placeholders (HMAC stable)
- ✓ Généralisation policy-driven (dates, orgs, IPs)
- ✓ Support paraphrase LLM (avec stub)
- ✓ Support RUPTA (avec stub)
- ✓ Audit + Hardening loop

Couche 3 - Évaluation

- ✓ Évaluateur avec métriques de base
- ✓ Validation des placeholders
- ✓ Warnings automatiques

2. Orchestrateur Refactorisé ✓

- ✓ Orchestrateur simplifié (~200 lignes)
- ✓ Injection de dépendances
- ✓ Gestion d'erreur globale
- ✓ Pas de logique métier (tout délégué aux services)

3. API Publique ✓

- ✓ Fonction `anonymize_text()` (API fonctionnelle)
- ✓ Classe `AnonymizationPipeline` (API orientée objet)
- ✓ Policy configurables avec presets L0/L1/L2
- ✓ Support batch processing

4. Documentation Complète ✓

- ✓ **README.md** : Vue d'ensemble et démarrage rapide
- ✓ **ARCHITECTURE.md** : Architecture détaillée des 3 couches
- ✓ **API_REFERENCE.md** : Documentation complète de l'API
- ✓ **QUICKSTART.md** : Exemples d'utilisation pas à pas

5. Tests & Exemples ✓

- ✓ Tests unitaires basiques (`tests/test_basic.py`)
- ✓ Exemples d'utilisation (`examples/simple_example.py`)

-  requirements.txt avec dépendances
-  .gitignore pour sécurité

6. Version Control ✓

-  Git initialisé
-  Commit initial avec message détaillé
-  37 fichiers, 5309 insertions, 3554 lignes de code



Métriques

Structure

31 fichiers Python
4 fichiers Markdown (documentation)
20 répertoires
3554 lignes de code Python

Réduction de Complexité

Métrique	Avant	Après	Amélioration
Lignes orchestrator	678	~200	-71%
Code dupliqué	RUPTA 2x	RUPTA 1x	-50%
NER code	872 (avec HF)	~600 (GLiNER)	-31%
Modules testables	3	10	+233%



Fonctionnalités Implémentées

Détection

-  Regex : 30+ patterns PII (email, phone, NIR, IBAN, dates, IPs, etc.)
-  NER GLiNER : 5 presets (fast, balanced, accuracy, pii, best)
-  GPU : Auto-detect CUDA/MPS/CPU avec fallback
-  Validation : IBAN, BIC, NIR (clé Luhn)

Transformation

-  Placeholders : Typed ([PER_ABC]) ou generic ([REDACTED])
-  Pseudonymisation : HMAC stable par scope
-  Généralisation : Dates (month/quarter/year), Orgs (generalize/redact)
-  Policy : 3 niveaux prédéfinis + customisation complète

Évaluation

-  Métriques : entities_detected, replacement_rate, length_ratio, etc.
-  Validation : placeholders well-formed, texte non vide
-  Warnings : taux de remplacement élevé, changement de taille

Configuration

Presets de Policy

L0 - Basic

- Regex + NER GLiNER uniquement
- Pas de LLM, pas de RUPTA
- Rapide et déterministe

L1 - Advanced

- L0 + LLM detection + paraphrase + audit
- RUPTA optimization
- Généralisation dates (month), orgs (generalize)

L2 - Maximum

- L1 avec généralisation très agressive
- Dates (year), orgs (redact)
- Paraphrase intensive

Variables d'Environnement

```
# GPU
export NER_FORCE_DEVICE=cuda # ou "mps", "cpu"
export NER_HALF_PRECISION=1 # FP16 sur CUDA

# GLiNER
export GLINER_PRESET=balanced # ou "fast", "accuracy", "pii", "best"
export GLINER_WEIGHTING=1 # Voting pondéré

# Debug
export NER_DEBUG=1 # Logs verbeux
```

Notes Importantes

Modules Stubs (À Compléter)

Les modules suivants sont des stubs et nécessitent le code complet :

1. **LLM Service** (`layer2_transformation/paraphrase/llm_reasoner.py`)
 - Copié depuis sources mais nécessite OpenRouter client complet
 - Fonctions : paraphrase, audit, detection avancée
2. **RUPTA** (`layer2_transformation/rupta/`)
 - Stubs créés mais logique complète à ajouter
 - Fichiers manquants : `optimizer.py` , `privacy_evaluator.py` , `utility_evaluator.py`
3. **OpenRouter Client**
 - Nécessaire pour les fonctionnalités LLM (L1+)
 - À copier depuis les sources originales

Tests

Les tests basiques sont fournis mais les tests complets nécessitent :

-  Tests unitaires par couche
-  Tests d'intégration end-to-end

- 🕒 Tests de performance (GPU vs CPU)
- 🕒 Tests RUPTA (nécessite implémentation complète)

Dépendances

Dépendances principales :

```
torch>=2.0.0      # NER + GPU
gliner>=0.1.0     # Modèles NER
schwifty>=2023.0.0 # IBAN/BIC (optionnel)
geonamescache>=1.5.0 # Villes (optionnel)
```

Pour LLM (L1+) :

```
openai>=1.0.0      # Si OpenAI
# ou autre provider LLM selon configuration
```



Utilisation Rapide

Test Basique

```
cd /home/ubuntu/anonymization_pipeline_refactored
python tests/test_basic.py
```

Exemples

```
python examples/simple_example.py
```

Import dans Votre Code

```
from api import anonymize_text, AnonymizationPipeline

# Fonction simple
result = anonymize_text(
    "Jean Dupont: jean@example.com",
    level="L0",
    secret_salt="my_secret"
)

# Pipeline réutilisable
pipeline = AnonymizationPipeline(level="L1", secret_salt="secret")
result = pipeline.anonymize("Mon texte sensible")
```



Documentation

Toute la documentation est dans docs/ :

- README.md - Vue d'ensemble
- docs/ARCHITECTURE.md - Architecture détaillée
- docs/API_REFERENCE.md - Référence API complète
- docs/QUICKSTART.md - Guide de démarrage

✓ Validation

Points Validés

- ✓ Structure en 3 couches claire
- ✓ Injection de dépendances fonctionnelle
- ✓ API publique simple et intuitive
- ✓ Documentation complète
- ✓ Tests basiques fonctionnels
- ✓ Git initialisé avec commit propre

À Valider en Production

- ⌚ Performance GPU réelle
- ⌚ Fonctionnalités LLM (nécessite setup complet)
- ⌚ RUPTA (nécessite implémentation complète)
- ⌚ Tests sur datasets réels

🎉 Conclusion

L'architecture refactorisée est **complète et fonctionnelle** pour les niveaux L0 (Regex + NER).

Les fonctionnalités L1+ (LLM, RUPTA) nécessitent l'ajout des modules manquants depuis les sources originales.

Prochaines étapes :

1. ✓ Tester avec `python tests/test_basic.py`
2. ✓ Explorer avec `python examples/simple_example.py`
3. ⌚ Ajouter les modules LLM/RUPTA complets si nécessaire
4. ⌚ Adapter à votre environnement de production

Version 2.0 - Architecture Refactorisée

Date : 3 Novembre 2025

Statut : ✓ Implémentation Complète (L0) / ⌚ Stubs (L1+)